

1. Datos del Proyecto

Nombre del proyecto	App web para la gestión de inventario y ventas de una farmacia local.
Cliente / Solicitante	Juan Pablo López.
Sector	Salud / Farmacéutico / Retail.
Descripción	Aplicación web orientada a distintos roles (administradores, cajeros, clientes) para gestionar inventario, ventas, pedidos en línea y clientes.
Tipo de despliegue	Contenedores orquestados (Docker / Kubernetes).
Plataforma	Web.
Escala de usuarios	100 – 10,000 usuarios.
Presupuesto total	Q 60,000.
Entorno	Cloud (Necesario para soportar clústeres de Kubernetes y la escala requerida).
Base de datos	Relacional (PostgreSQL o MySQL).
Metodología	XP (Extreme Programming).
Equipo	7.

2. Requerimientos No Funcionales

Rendimiento

Tiempo de respuesta promedio	Menor a 1 segundo. Esencial para garantizar una atención rápida por parte de los cajeros y una navegación fluida para los clientes web.
Latencia máxima (p95)	2.0 a 3.0 segundos máximo, considerando el tiempo extra que toma la comunicación mediante eventos entre el microservicio de Ventas y el de Inventario para actualizar el stock.

Estrategia de escalado	Horizontal y Dinámico. Utilización de la orquestación de Kubernetes para multiplicar automáticamente los contenedores de los microservicios con mayor demanda (ej. Ventas) durante picos de tráfico.
Crecimiento proyectado	Alto. La arquitectura está diseñada para soportar un crecimiento escalonado desde 100 hasta 10,000 usuarios concurrentes sin pérdida de rendimiento.

Disponibilidad

Disponibilidad requerida	Alta disponibilidad (ej. 99.9%). Esencial para que las ventas y el control de inventario de la farmacia no se detengan, soportando picos de hasta 10,000 usuarios.
RTO	Muy bajo (minutos o segundos). Al usar contenedores y Kubernetes, si un servicio falla, el orquestador reinicia o levanta una nueva réplica casi de inmediato.
RPO	Bajo. Requiere políticas de respaldo continuo en las bases de datos relacionales (PostgreSQL/MySQL) de cada microservicio para no perder registros de ventas o clientes.
Estrategia ante fallos	Auto-healing (Autocorrección). Kubernetes monitorea el estado de los contenedores; si un microservicio (como Ventas o Inventario) cae, aísla el fallo y levanta una nueva instancia sin afectar al resto del sistema.

Seguridad

Datos sensibles	Información personal de los clientes, su historial de compras/medicamentos, credenciales de usuarios y registros financieros de ventas.
Normativas de referencia	(Inferencia arquitectónica) Buenas prácticas para protección de datos médicos/personales (ej. HIPAA o equivalente local) y OWASP para asegurar el API Gateway.
Autenticación	Centralizada mediante el API Gateway, el cual valida la identidad del usuario antes de enrutar la petición hacia los servicios internos.

Autorización	Control de accesos basado en roles (Administradores, Cajeros, Clientes). Gestionado en conjunto por el API Gateway y el Microservicio de Usuarios (Identidad y Seguridad).
Encriptación en tránsito	Uso de protocolos seguros (HTTPS/TLS) para toda la comunicación entre la aplicación web frontend y el API Gateway.
Encriptación en reposo	Cifrado a nivel de disco o de base de datos en las instancias de PostgreSQL/MySQL que guardan los datos críticos de Clientes, Usuarios y Ventas.

3. Arquitectura de Software

Estilo arquitectónico

Patrón elegido	Arquitectura de Microservicios.
Justificación	Esta estructura permite soportar de manera eficiente y dinámica la escala requerida (de 100 a 10,000 usuarios). Al dividir la lógica en servicios independientes, facilita el escalamiento individual de módulos críticos (como Ventas) y apoya los lanzamientos rápidos y entregas iterativas que exige la metodología Extreme Programming (XP).

Módulos del sistema (ejemplos provistos)

Módulo	Responsabilidad	Expone API	Tecnología
API Gateway	Actúa como único punto de entrada; enruta las peticiones a los microservicios y valida la seguridad/autenticación inicial.	Sí	NGINX, Kong o AWS API Gateway
Inventario y Medicamentos	Registro de medicamentos (nombre, precio, stock) y control de inventario en tiempo real para evitar ventas sin existencias.	Sí	Node.js, Java o Python

Ventas	Procesa el registro de ventas y pedidos en línea de forma rápida y segura. Emite eventos para actualizar el stock.	Sí	Node.js, Java o Python
Clientes	Gestiona la información de los clientes y permite consultar su historial de compras para análisis comercial.	Sí	Node.js, Java o Python
Reportes	Extrae y consolida información del comportamiento de productos e historial de ventas para la toma de decisiones.	Sí	Node.js, Java o Python
Usuarios (Autenticación)	Administra las cuentas de los usuarios y gestiona los roles (administradores, cajeros, clientes) para controlar accesos.	Sí	Node.js, Java o Python

4. Stack Tecnológico

Capa	Tecnología	Versión	Justificación
Backend / API	Node.js (Express o NestJS) / Java / Python	LTS	Permite desarrollar microservicios independientes, escalables y orientados a eventos para manejar altas cargas transaccionales.
Frontend Web	React.js o Vue.js	18.x / 3.x	Facilita la creación de una interfaz dinámica, rápida y responsiva para los diferentes roles (cajeros, clientes, administradores).
ORM	Prisma o Sequelize (Si es Node) / Hibernate (Si es Java)	LTS	Simplifica el manejo y las migraciones de los esquemas de bases de datos relacionales independientes de cada microservicio.
Base de Datos	PostgreSQL o MySQL	15.x / 8.x	Motores relacionales robustos. Se usará el patrón "Database-per-service", donde cada microservicio gestiona su propio esquema.
Autenticación	JWT (jsonwebtoken)	—	Estándar apátrida (stateless) ideal para microservicios, permitiendo al API Gateway validar tokens de

			usuarios sin consultar constantemente la BD.
Recordatorios	Nodemailer + cron	—	Útil para enviar notificaciones asíncronas, como alertas de inventario bajo a los administradores o confirmaciones de pedidos a los clientes.
Contenedores	Docker (+ Kubernetes)	Latest	Empaqueta cada microservicio para asegurar consistencia entre entornos y permite que Kubernetes escale las instancias hasta soportar 10,000 usuarios.
CI/CD	GitHub Actions / GitLab CI	—	Automatiza la ejecución de pruebas unitarias y la integración diaria, cumpliendo con los principios de entregas rápidas de Extreme Programming (XP).
Servidor web	NGINX o Kong (Como API Gateway)	Latest	Actúa como puerta de enlace principal: recibe el tráfico web, valida la seguridad inicial y enruta las peticiones al contenedor correspondiente.

5. Infraestructura

Componentes de red y enrutamiento

Componente	Aplica	Herramienta / Servicio	Notas
Proxy inverso	Sí	NGINX Ingress Controller	Gestiona el tráfico entrante hacia el clúster de Kubernetes, exponiendo los servicios internos de forma segura hacia el exterior.
Balanceador de carga	Sí	Cloud Load Balancer (ej. AWS ELB)	Crítico para distribuir el tráfico masivo (hasta 10,000 usuarios) de manera equitativa entre los

			múltiples nodos y réplicas de los contenedores.
API Gateway	Sí	Kong, NGINX o AWS API Gateway	Punto único de entrada definido en la arquitectura; maneja el enrutamiento hacia los microservicios y valida la autenticación inicial
CDN	Sí	Cloudflare o Amazon CloudFront	Recomendado para la aplicación web (React/Vue). Acelera la carga de la interfaz de usuario sirviendo los archivos estáticos desde ubicaciones geográficas cercanas, reduciendo la latencia.
WAF / Firewall de aplicación	Sí	Cloudflare WAF o AWS WAF	Protege el API Gateway y la aplicación web contra vulnerabilidades (OWASP Top 10), garantizando la seguridad de los datos sensibles de ventas y clientes de la farmacia.
Red privada (VPC)	Sí	AWS VPC (Virtual Private Cloud) o equivalente	Aísla los nodos del clúster de Kubernetes y las bases de datos relacionales de la internet pública. El acceso a los microservicios solo es posible a través del API Gateway.

Almacenamiento

Tipo	Aplica	Servicio	Notas
Disco persistente (EBS)	Sí	AWS EBS (Elastic Block Store)	Fundamental para proveer volúmenes persistentes a los contenedores de las bases de datos relacionales (PostgreSQL/MySQL) en Kubernetes. Asegura que los datos críticos de ventas e inventario no

			se pierdan si un contenedor se reinicia.
Object Storage (archivos)	Sí	AWS S3 (Simple Storage Service)	Ideal para almacenar recursos estáticos, imágenes de los medicamentos, o documentos generados por el sistema (como facturas o reportes de ventas en PDF), manteniendo las bases de datos ligeras y optimizadas.

Servidores y capacidad

Componente	Instancias	Spec / Tipo	Notas
Servidor de aplicación	Auto-escalable (Múltiples Nodos)	Nodos de Kubernetes (ej. AWS EC2 - t3.medium / m5.large)	Nodos del clúster que alojan los contenedores de los microservicios. Se configuran con auto-escalamiento horizontal para soportar dinámicamente el tráfico de 100 a 10,000 usuarios.
Base de Datos	Múltiples (Una por microservicio)	Servicios administrados (ej. AWS RDS PostgreSQL / MySQL)	Se implementa el patrón de una base de datos por servicio. Cada instancia administra su propio esquema de forma independiente (Ventas, Inventario, Clientes, etc.) para evitar cuellos de botella.
Servidor de staging	1 Clúster replicado (Reducido)	Nodos de menor capacidad (Cloud) o entorno de pruebas local	Entorno idéntico a producción pero a menor escala. Indispensable para que los pipelines de CI/CD ejecuten pruebas continuas y validen las iteraciones cortas de la metodología

			XP antes del despliegue final.
--	--	--	--------------------------------

6. Base de Datos

Diseño provisto por el área de BD. Diagrama ER disponible en Lucidchart (entregable externo).

Motor	PostgreSQL o MySQL.
Tipo	Relacional, aplicando el patrón de "Database-per-service" (Base de datos por servicio).
Justificación	Para respetar la arquitectura de microservicios, cada uno debe gestionar su propio esquema. Esta separación es clave para soportar de manera eficiente la escala requerida de 100 a 10,000 usuarios sin crear cuellos de botella centralizados.
Entidades principales	Medicamento, Inventario, Venta, Cliente, Usuario y Rol.
Despliegue	Múltiples esquemas independientes , los cuales pueden integrarse en servicios administrados en la nube o dentro del entorno de contenedores.
Backup	Respaldos automatizados y segregados por cada esquema independiente de los microservicios.
ORM / Migraciones	Automatizadas a través de los pipelines de Integración Continua (CI/CD) para asegurar que se integren diariamente y se adapten a las iteraciones cortas requeridas por la metodología XP.
Acceso	Restringido y privado; cada esquema es accedido exclusivamente por su microservicio correspondiente. Las aplicaciones cliente acceden a los datos únicamente pasando primero por el API Gateway.

7. Seguridad

Área	Implementación
Autenticación	Centralizada a través del API Gateway utilizando tokens JWT (JSON Web Tokens) generados por el microservicio de Usuarios.
Autorización	Control de Acceso Basado en Roles (RBAC) para diferenciar permisos entre administradores, cajeros y clientes, validado a través de los claims del JWT.
Contraseñas	Almacenamiento seguro utilizando algoritmos de hashing fuertes (como bcrypt o Argon2) dentro de la base de datos del microservicio de Usuarios; nunca en texto plano.
Gestión de secretos	Uso de Kubernetes Secrets (o AWS Secrets Manager) para inyectar credenciales de bases de datos y claves privadas en los contenedores, evitando codificarlos en el código fuente.
Rate limiting	Configurado en el API Gateway para limitar la cantidad de peticiones por usuario/IP, previniendo ataques de denegación de servicio (DDoS) y ataques de fuerza bruta.
CORS	Políticas estrictas configuradas en el API Gateway para aceptar peticiones únicamente desde los dominios autorizados donde se aloje la aplicación web frontend (React/Vue).
Validación de inputs	Sanitización y validación estricta de datos en la entrada de cada microservicio para prevenir inyección SQL, Cross-Site Scripting (XSS) y asegurar la integridad de precios e inventario.
HTTPS	Cifrado en tránsito obligatorio. El tráfico externo se maneja mediante TLS (SSL) asegurado desde el proxy inverso / API Gateway hasta el cliente final.
Aislamiento de BD	Las bases de datos operan dentro de una Red Privada (VPC). No tienen exposición directa a Internet; solo el microservicio correspondiente puede comunicarse con su respectiva base de datos.
Logs de auditoría	Trazabilidad centralizada. Se registran eventos críticos (quién registró una venta, quién modificó el stock de un medicamento) para el análisis de comportamiento y auditorías de seguridad.

8. Contenedores y Despliegue

Estrategia de contenedores	Empaquetado individual de cada microservicio utilizando Docker. Su orquestación se maneja mediante Kubernetes para garantizar el lanzamiento rápido de iteraciones (alineado a XP) y el escalamiento masivo para soportar hasta 10,000 usuarios.
Registry de imágenes	Repositorio privado en la nube como Amazon ECR (Elastic Container Registry) o Docker Hub. Almacena de forma segura las imágenes generadas por el pipeline de CI/CD para cada microservicio.
Proxy / Servidor web	NGINX Ingress Controller en el clúster de Kubernetes. Trabaja en conjunto con el API Gateway para recibir el tráfico web y enrutarlo hacia los Pods correspondientes a cada servicio interno (Ventas, Inventario, etc.).
Health checks	Configuración de Liveness y Readiness Probes en Kubernetes. El orquestador verifica constantemente la salud de cada microservicio; si el servicio de Ventas o Inventario falla, reinicia el contenedor automáticamente sin afectar el resto del sistema.
Estrategia de deploy	Rolling Updates nativos de Kubernetes. Permite desplegar nuevas versiones de los microservicios reemplazando los contenedores de forma gradual, logrando cero tiempo de inactividad (zero downtime) durante las entregas continuas que exige la metodología XP.
Ambientes	Desarrollo (Local con Docker Compose), Staging (Clúster de K8s reducido para validar la integración diaria) y Producción (Clúster principal auto-escalable).
Rollback	Reversión inmediata a través de Kubernetes (kubectl rollout undo). Si una nueva versión introducida en una iteración corta falla, el sistema regresa automáticamente a la imagen del contenedor de la versión anterior estable.

9. CI/CD

Plataforma	GitLab o GitHub (Es altamente recomendado tener un repositorio independiente para cada microservicio).
Estrategia de branching	Trunk-Based Development. Ideal para Extreme Programming (XP), ya que fomenta que los desarrolladores integren pequeñas piezas de código directamente a la rama principal (main) de forma continua, evitando conflictos grandes.
Herramienta CI/CD	GitLab CI/CD o GitHub Actions. Proveen los pipelines automatizados necesarios para soportar las iteraciones cortas y asegurar que el código se integre diariamente.
Etapas del pipeline	1. Lint/Build: Verificación de sintaxis y compilación. 2. Test: Ejecución de pruebas unitarias continuas (requisito indispensable en XP antes de cualquier entrega). 3. Dockerize: Construcción de la imagen Docker y subida al Registry. 4. Deploy: Actualización de los Pods en Kubernetes.
Triggers	Ejecución automática configurada al hacer Push directo a la rama main (integración diaria) o al abrir/actualizar un Pull Request.
Convención de commits	Conventional Commits. Uso de prefijos estandarizados (ej. feat, fix, test:) para mantener un historial limpio, lo cual es crítico cuando múltiples desarrolladores tocan distintos repositorios de microservicios.

10. Observabilidad

Área	Aplica	Herramienta	Descripción
Logging	Sí	ELK Stack (Elasticsearch, Logstash, Kibana) o CloudWatch	Centraliza los logs de todos los microservicios. En lugar de revisar contenedor por contenedor, permite buscar errores en un solo lugar si falla, por ejemplo, el registro de una venta.
Métricas de infra	Sí	Prometheus	Recolecta métricas en tiempo real del clúster de Kubernetes (CPU, memoria, red). Esencial para que el auto-escalado funcione correctamente al acercarse a los 10,000 usuarios.
Tracing distribuido	Sí	Jaeger o AWS X-Ray	Vital en microservicios. Permite rastrear la ruta exacta de una petición (ej. API Gateway -> Ventas -> Inventario) para descubrir en qué punto exacto ocurre la lentitud o el fallo de un evento.
Dashboards	Sí	Grafana (o Kibana)	Se conecta con Prometheus para crear paneles visuales e interactivos. Permite monitorear el estado de salud del sistema, la latencia de las APIs y lanzar alertas en tiempo real.

11. Costos Estimados de Infraestructura

Componente	Servicio	Tier / Tipo	USD / mes	Notas
Servidor de aplicación	AWS EKS + EC2 Nodes	EKS Cluster + 3x Nodos t3.large	\$250.00	Aloja el clúster de Kubernetes que orquesta los microservicios. Auto-escalable para soportar los picos de hasta 10,000 usuarios.
Base de Datos	AWS RDS (PostgreSQL)	Multi-AZ db.t3.medium	\$150.00	Instancias de base de datos divididas lógicamente por microservicio (Inventario, Ventas, Usuarios) para asegurar alta disponibilidad.
Servidor de staging	AWS EC2	1x Nodo t3.medium	\$30.00	Entorno replicado a menor escala para ejecutar las pruebas continuas de los pipelines CI/CD exigidas por la metodología XP.
Almacenamiento	AWS S3 + AWS EBS	Standard + gp3 (Discos)	\$40.00	Volúmenes EBS para persistencia de bases de datos y S3 para almacenamiento

				de recursos estáticos (imágenes de medicamentos, reportes).
Red / Transferencia	AWS ALB + CloudFront	Application Load Balancer	\$50.00	Balanceador de carga esencial para distribuir el tráfico masivo de usuarios entre los diferentes contenedores de la aplicación web.
Registry de imágenes	Amazon ECR	Almacenamiento Estándar	\$10.00	Repositorio privado y seguro para almacenar las imágenes Docker de cada uno de los microservicios generados.
Monitoring / Logs	AWS CloudWatch	Standard Metrics/Logs	\$30.00	Centraliza los logs de todos los microservicios y monitorea la salud del clúster para disparar el auto-escalado.
Certificado SSL	AWS Certificate Manager	Integrado con Load Balancer	\$0.00	Renovación automática y gratuita para asegurar todo el tráfico web bajo HTTPS.
Total mensual estimado			\$560.00	

Presupuesto total del proyecto	Q 60,000.00
Infra mensual estimada	Q 4,368.00 (\$560 USD)
Infra por 16 semanas (4 meses)	Q 26,208.00
Observación	Si el proyecto requiere 6 meses de operación/despliegue inicial, la infraestructura consumirá ~43% del presupuesto, dejando un amplio margen (Q 33,792) para cubrir costos de desarrollo, licencias y honorarios del equipo bajo la metodología XP.

12. Decisiones Arquitectónicas

ID	Área	Opción Elegida	Alternativa	Justificación
ADR-01	Patrón arquitectónico	Microservicios	Arquitectura Monolítica	Permite el escalado independiente de módulos críticos (como Ventas) para soportar hasta 10,000 usuarios concurrentes.
ADR-02	Framework backend	Node.js / Java / Python (Contenedores)	Frameworks de un solo lenguaje (ej. PHP)	Ofrece flexibilidad para elegir la mejor herramienta según el microservicio y facilita el despliegue rápido exigido por XP.
ADR-02b	Framework frontend	React / Vue.js (Web Cliente)	Aplicación de escritorio o SSR	Proporciona una interfaz ágil para cajeros y clientes, consumiendo servicios de forma asíncrona a través del API Gateway.
ADR-03	ORM	Prisma / Hibernate (Database-per-service)	Una sola base de datos centralizada	Garantiza el aislamiento de datos; cada microservicio gestiona su propio esquema (PostgreSQL/MySQL) de forma independiente.

ADR-04	Despliegue en cloud	Kubernetes (Orquestación)	Máquinas virtuales individuales (IaaS)	Necesario para automatizar el auto-escalado horizontal, la salud de los contenedores (health checks) y las iteraciones cortas de XP.
ADR-05	Sin balanceador de carga	Load Balancer + API Gateway	Sin balanceador de carga	Mandatorio. Dado el volumen de hasta 10,000 usuarios, es indispensable distribuir el tráfico y centralizar la autenticación antes de llegar a los servicios.
ADR-06	Recordatorios con cron	Worker especializado / Eventos	Tareas manuales o triggers de base de datos	Permite manejar notificaciones (ej. stock bajo) sin bloquear la lógica principal, comunicándose de forma asíncrona mediante eventos.

13. Diagrama de Arquitectura

